
Sports Event Management

MVP Master Specification Document

Full System Blueprint · Coding Agent Prompt · Technical Reference

SE Team 026 – Syntax Error

February 17, 2026 | Version 1.0

Backend Stack:	Flask, Flask-RESTful, SQLAlchemy, PostgreSQL
Frontend Stack:	Vue 3 CLI, Vue Router, Pinia, JavaScript
GenAI:	OpenAI, Gemini, HuggingFace, LangChain
Search:	Algolia
Payments:	Razorpay
Auth:	JWT (Flask-JWT-Extended)
VCS / PM:	GitHub / Jira
Total Features:	18 Intelligence Features
Sprint:	4 Weeks / $\approx$ 20 Developer Days

▷ **Coding Agent Notice:** This document is both a technical specification and a structured master prompt. Every section is written as a direct instruction. Build exactly what is described. No feature should be omitted. Follow the role-based access rules, data models, API contracts, and UI flows precisely as documented.

Contents

1	Project Overview & Agent Instructions	4
1.1	Purpose of This Document	4
1.2	Problem Statement	4
1.3	Application Entry Point & Navigation Flow	4
2	Technology Stack	6
2.1	Backend Technologies	6
2.2	Frontend Technologies	6
2.3	GenAI Technologies	6
2.4	Third-Party Services	6
3	Data Models & Relationships	8
3.1	Entity Relationship Summary	8
3.2	Model 1: User	8
3.3	Model 2: Event	9
3.4	Model 3: Registration	10
3.5	Model 4: Payment	11
3.6	Model 5: EscalationTicket (Chatbot Fallback)	12
4	User Roles & Access Control	13
4.1	Role Definitions	13
4.1.1	Role: user	13
4.1.2	Role: organizer	13
4.1.3	Role: admin	13
4.2	Post-Login Routing Rules	14
5	Frontend Pages & UI Specification	15
5.1	Page 1: Landing Page (/)	15
5.2	Page 2: Register Page (/register)	15
5.3	Page 3: Login Page (/login)	16
5.4	Page 4: User Home – Event Discovery (/home)	16
5.5	Page 5: Event Detail Page (/event/:id)	16
5.6	Page 6: Organizer Dashboard (/organizer)	16
5.7	Page 7: Admin Dashboard (/admin)	17
5.8	Component: AI Chatbot Widget	17
6	API Endpoint Specification	18
6.1	Authentication Endpoints	18
6.2	Event Endpoints	19
6.3	Registration Endpoints	20

6.4	Payment Endpoints	21
6.5	Chatbot Endpoint	22
6.6	Organizer Dashboard Endpoints	23
6.7	Admin Analytics Endpoints	25
7	Intelligence Features Specification	26
7.1	User Features (1–7)	26
7.1.1	Feature 1 – Event Recommendation by Preferred Sports	26
7.1.2	Feature 2 – Nearby Events Recommendation	26
7.1.3	Feature 3 – Upcoming Events Recommendation	26
7.1.4	Feature 4 – Budget-Based Recommendation	26
7.1.5	Feature 5 – Similar Event Recommendation	26
7.1.6	Feature 6 – Razorpay Payment Gateway	27
7.1.7	Feature 7 – AI Chatbot	27
7.2	Organizer Features (8–12)	27
7.2.1	Feature 8 – Organizer Event Dashboard	27
7.2.2	Feature 9 – Event Performance Label	27
7.2.3	Feature 10 – Registration Trend Graph	27
7.2.4	Feature 11 – Ticket Sales Summary	27
7.2.5	Feature 12 – Popular Category Insight	28
7.3	Admin Features (13–18)	28
7.3.1	Feature 13 – Platform Analytics Dashboard	28
7.3.2	Feature 14 – Most Popular Sport Category	28
7.3.3	Feature 15 – City-Wise User Distribution	28
7.3.4	Feature 16 – Event Fill-Rate Dashboard	28
7.3.5	Feature 17 – Monthly Registration Trend	28
7.3.6	Feature 18 – Organizer Performance Comparison	28
8	Algolia Search Specification	29
8.1	Algolia Index Schema	29
8.2	Search Configuration	29
8.3	Frontend Search Behaviour	29
9	Payment Flow Specification	30
9.1	End-to-End Payment Flow	30
9.2	Platform Fee Calculation	30
10	AI Chatbot Specification	31
10.1	RAG Pipeline Architecture	31
10.2	Initial FAQ Knowledge Base	31
11	Folder Structure & Project Setup	33
11.1	Backend Directory Layout	33
11.2	Frontend Directory Layout	33
11.3	Environment Variables	34
12	Implementation Sprint Plan	35
12.1	Sprint Overview	35
12.2	Feature Effort Summary	36

13 Hosting & Deployment	38
13.1 Hosting Options	38
13.2 Deployment Checklist	38
Conclusion & Agent Summary	39

1. Project Overview & Agent Instructions

1.1 Purpose of This Document

This document is the **single source of truth** for building the Sports Event Management MVP. It serves two simultaneous purposes:

1. **Technical Specification** – Every model, relationship, API endpoint, page, component, and feature is documented in full detail without ambiguity.
2. **Coding Agent Master Prompt** – A software engineering AI agent must read this document top-to-bottom and implement every instruction contained within it. Nothing should be assumed or skipped.

Agent Rule #1: You are building a full-stack web application. The backend must be built with Flask and Flask-RESTful. The frontend must be built with Vue 3. Every data model, API endpoint, and UI page listed in this document must be implemented completely and correctly.

1.2 Problem Statement

The sports event industry lacks a unified digital platform that efficiently connects event organizers with participants while providing intelligent recommendations, real-time analytics, and seamless payment processing. Organizers struggle to monitor event performance, administrators lack platform-wide visibility, and users face difficulty discovering relevant local events.

This platform solves all three problems by delivering a single web application with three distinct role-based interfaces, intelligent recommendation logic, BI dashboards, Razorpay payments, Algolia search, and a LangChain-powered AI chatbot.

1.3 Application Entry Point & Navigation Flow

Agent Rule #2: The very first page a visitor sees when opening the application must be the **Landing Page**. The landing page is a public marketing page. It must

contain a navigation bar with a **Login** button and a **Register** button. No dashboard or event data should be visible before the user authenticates.

The complete navigation flow of the application is as follows:

Step	Page	Who Sees It
1	Landing Page	All visitors (public)
2	Register Page	New visitors
2	Login Page	Returning users
3a	User Home / Event Discovery	Role = user
3b	Organizer Dashboard	Role = organizer
3c	Admin Dashboard	Role = admin

2. Technology Stack

2.1 Backend Technologies

The backend is a RESTful API server built on the Python ecosystem.

Flask is the core web framework. **Flask-RESTful** is used to define all API resources as class-based views with clean HTTP method separation. **SQLAlchemy** is the ORM layer; it maps Python classes to database tables and manages all queries. **Flask-Migrate** (built on Alembic) handles all database schema migrations. **Flask-JWT-Extended** manages JSON Web Token authentication — every protected endpoint verifies a Bearer token in the Authorization header. **Flask-CORS** enables cross-origin requests from the Vue frontend. The development database is **SQLite** and the production database is **PostgreSQL**.

2.2 Frontend Technologies

The frontend is a Single Page Application (SPA) built with **Vue 3 CLI**. **Vue Router** handles client-side routing between pages without full page reloads. **Pinia** is the global state management library — it stores the authenticated user's token, role, and ID across all components. **Axios** is the HTTP client used to call all backend API endpoints. **Chart.js** with the **vue-chartjs** wrapper renders all analytics charts (line graphs and bar charts). The **Razorpay JavaScript SDK** is loaded via a CDN script tag to open the payment checkout modal.

2.3 GenAI Technologies

The AI chatbot is built using **LangChain** as the orchestration framework. LangChain connects to **OpenAI GPT-4o-mini** as the primary LLM. As fallback options, **Google Gemini** and **HuggingFace** inference models may also be used. A **FAISS** vector store holds the FAQ knowledge base as embeddings. The chatbot uses a **Retrieval-Augmented Generation (RAG)** pipeline: incoming queries are matched against the vector store and the top matching FAQ chunks are injected into the LLM prompt as context.

2.4 Third-Party Services

Algolia provides instant, typo-tolerant full-text search over all events. Event records are synced to Algolia any time an event is created or updated. The Vue frontend uses the Algolia JavaScript client directly for search queries.

Razorpay handles all payment processing — order creation, checkout modal, webhook-based payment verification, and organizer payout calculations with platform fee deduction.

GitHub is used for version control, pull requests, code reviews, and issue tracking. **Jira** is used for sprint and project management.

3. Data Models & Relationships

Agent Rule #3: Implement all five database models exactly as described below. Every field, data type, constraint, and relationship must be created. Use SQLAlchemy ORM classes. Run Flask-Migrate to generate and apply migrations.

3.1 Entity Relationship Summary

The five core models and their relationships are:

- A **User** can have many **Registrations** (one-to-many).
- An **Event** can have many **Registrations** (one-to-many).
- A **Registration** belongs to exactly one User and one Event (many-to-one each).
- A **Registration** has exactly one **Payment** (one-to-one).
- A **User** with role **organizer** can own many **Events** (one-to-many).

From Model	Relationship	To Model
User (organizer)	has many	Event
User	has many	Registration
Event	has many	Registration
Registration	belongs to	User
Registration	belongs to	Event
Registration	has one	Payment

3.2 Model 1: User

The User model represents all three roles in a single table, differentiated by the **role** field.

Field	Type	Description	Constraints
id	Integer	Primary key, auto-increment	PK, Not Null
name	String(100)	Full name of the user	Not Null
email	String(150)	Email used for login	Unique, Not Null
password_hash	String(256)	Bcrypt/Werkzeug hashed password	Not Null
role	String(20)	One of: <code>user</code> , <code>organizer</code> , <code>admin</code>	Default: <code>user</code>
city	String(100)	User's home city for nearby events	Nullable
budget_preference	String(20)	One of: <code>cheap</code> , <code>mid</code> , <code>premium</code>	Default: <code>mid</code>
preferred_sports	JSON	List of sport names, e.g. <code>["football", "tennis"]</code>	Nullable
created_at	DateTime	Timestamp of account creation	Server default NOW()

Relationships: User has many Registrations (back-populated as `registrations`). Organizer-role Users have many Events (back-populated as `events_organized`).

Methods required: `set_password(plain_text)` hashes and stores the password. `check_password(p)` verifies the password against the hash.

3.3 Model 2: Event

The Event model represents a sports event created by an organizer.

Field	Type	Description	Constraints
id	Integer	Primary key, auto-increment	PK, Not Null
title	String(200)	Name of the event	Not Null
sport_category	String(100)	Sport type, e.g. <code>football</code> , <code>tennis</code>	Not Null
description	Text	Full description of the event	Nullable
venue_city	String(100)	City where event takes place	Nullable
venue_address	String(300)	Full venue address	Nullable
event_date	DateTime	Date and time of the event	Not Null
capacity	Integer	Maximum number of participants	Not Null
price	Float	Ticket price in INR	Not Null
price_tier	String(20)	Derived tier: <code>cheap</code> , <code>mid</code> , <code>premium</code>	Nullable
organizer_id	Integer	Foreign key to users.id	FK, Not Null
tags	JSON	List of descriptive tags, e.g. <code>["outdoor", "team"]</code>	Nullable
banner_url	String(300)	URL to event banner image	Nullable
is_active	Boolean	Whether event is publicly visible	Default: True
created_at	DateTime	Timestamp of event creation	Server default NOW()

Computed Properties (not stored in DB):

- `seats_sold` — Count of Registrations with status `confirmed` for this event.
- `seats_remaining` — `capacity` minus `seats_sold`.
- `fill_rate` — `seats_sold / capacity × 100`, rounded to 1 decimal.
- `performance_label` — Returns `LOW` if fill rate < 30%, `MEDIUM` if 30–70%, `HIGH` if > 70%.
- `revenue` — `seats_sold × price`.

Price Tier Logic: When an event is saved, automatically assign `price_tier` based on the `price` field. Cheap is under 500, Mid is 500–2000, Premium is above 2000. These thresholds must be configurable from an admin setting or environment variable.

3.4 Model 3: Registration

A Registration record is created each time a user registers for an event.

Field	Type	Description	Constraints
id	Integer	Primary key, auto-increment	PK, Not Null
user_id	Integer	Foreign key to users.id	FK, Not Null
event_id	Integer	Foreign key to events.id	FK, Not Null
status	String(30)	One of: <code>pending</code> , <code>confirmed</code> , <code>cancelled</code>	Default: <code>pending</code>
created_at	DateTime	Timestamp of registration creation	Server default NOW()

Relationships: Registration belongs to User and Event. Registration has one Payment (back-populated as `payment`).

Constraint: A user must not be able to register for the same event twice. Enforce a unique constraint on the combination of (`user_id`, `event_id`).

3.5 Model 4: Payment

A Payment record tracks the Razorpay transaction associated with a Registration.

Field	Type	Description	Constraints
id	Integer	Primary key, auto-increment	PK, Not Null
registration_id	Integer	Foreign key to registrations.id	FK, Unique
razorpay_order_id	String(100)	Order ID returned by Razorpay	Unique
razorpay_payment_id	String(100)	Payment ID after successful payment	Nullable
razorpay_signature	String(300)	Signature string for verification	Nullable
amount	Float	Total amount paid in INR	Not Null
platform_fee	Float	Platform deduction (15–20% of amount)	Not Null
organizer_payout	Float	Amount = amount minus platform_fee	Not Null
status	String(30)	One of: <code>created</code> , <code>paid</code> , <code>failed</code>	Default: <code>created</code>
created_at	DateTime	Timestamp of payment initiation	Server default NOW()

3.6 Model 5: EscalationTicket (Chatbot Fallback)

When the AI chatbot cannot answer a query, an escalation ticket is logged.

Field	Type	Description	Constraints
id	Integer	Primary key, auto-increment	PK
user_query	Text	The original message from the user	Not Null
session_context	Text	Recent conversation history for context	Nullable
is_resolved	Boolean	Whether admin has responded	Default: False
created_at	DateTime	Timestamp of escalation	Server default NOW()

4. User Roles & Access Control

Agent Rule #4: Every backend API endpoint must enforce role-based access control. A custom decorator called `role_required(role)` must be implemented and applied to every protected route. If a user's JWT role does not match the required role, return HTTP 403 Forbidden.

4.1 Role Definitions

The platform has exactly three user roles, all stored in the `role` field of the User model:

4.1.1 Role: user

A standard end-user who discovers and registers for events. After login, they are redirected to the **Home / Event Discovery** page. They can browse recommended events, search events via Algolia, view event details, register for events, pay via Razorpay, and use the AI chatbot.

4.1.2 Role: organizer

An event organizer who creates and manages sports events. After login, they are redirected to the **Organizer Dashboard**. They can create, edit, and delete their own events. They can also view their event performance metrics, registration trends, ticket sales summaries, and category insights. They cannot see other organizers' data and cannot access admin features.

4.1.3 Role: admin

A platform administrator. After login, they are redirected to the **Admin Dashboard**. They have read access to all platform data including total users, all events, all registrations, city distributions, monthly trends, and organizer performance comparisons. They also receive and resolve chatbot escalation tickets. They cannot be registered through the public registration form — admin accounts must be seeded directly into the database.

4.2 Post-Login Routing Rules

JWT Role Claim	Redirect To	Page
user	/home	Event Discovery & Recommendations
organizer	/organizer	Organizer Dashboard
admin	/admin	Admin Analytics Dashboard

5. Frontend Pages & UI Specification

Agent Rule #5: Build every page listed in this chapter as a Vue 3 single-file component (.vue). Use Vue Router for navigation. Use Pinia for shared state. Each page must be fully functional — no placeholder or lorem ipsum content.

5.1 Page 1: Landing Page (/)

This is the **first page every visitor sees**. It is a public marketing page that requires no authentication.

Navbar: Contains the platform logo/name on the left. On the right side, display two buttons: **Login** (navigates to /login) and **Register** (navigates to /register).

Hero Section: A large, visually striking full-width section with a headline such as *"Discover. Register. Play."*, a subheading describing the platform in one sentence, and a call-to-action button **Get Started** that navigates to /register.

Features Section: Three or four highlight cards showing the platform's key value propositions — smart recommendations, easy registration, secure payments, and 24/7 AI support.

How It Works Section: A numbered three-step flow — Sign Up, Discover Events, Register & Pay.

Footer: Platform name, team name (SE Team 026 – Syntax Error), and copyright.

5.2 Page 2: Register Page (/register)

A registration form that collects the following fields from the new user:

Full Name, Email Address, Password, Confirm Password, Role (dropdown: **User** or **Organizer** — Admin cannot be registered publicly), City, Budget Preference (dropdown: Cheap / Mid / Premium), and Preferred Sports (multi-select checkboxes for common sports: Football, Cricket, Tennis, Basketball, Running, Badminton, Swimming, Cycling).

On successful registration, the user receives a JWT and is redirected based on their role.

5.3 Page 3: Login Page (/login)

A login form that collects Email and Password. On successful authentication, the JWT is stored in the Pinia auth store and the user is redirected based on their role claim inside the JWT. All three roles (user, organizer, admin) log in through this same page.

5.4 Page 4: User Home – Event Discovery (/home)

This page is accessible only to authenticated users with role **user**.

Search Bar: A prominent Algolia-powered search bar at the top. As the user types, results appear instantly below without page reload.

Recommended Events Section: A horizontally scrollable row of event cards. The events shown here are personalised based on the user's preferred sports, city, budget, and upcoming dates (Features 1–4 from the BI proposal).

This Week Section: A separate row showing only events happening within the next 7 days (Feature 3).

Nearby Events Section: A row of events in the user's registered city (Feature 2).

All Events Grid: A paginated grid of all active events sorted by date.

Each **Event Card** must display: event title, sport category badge, city, date, price, and a "View Details" button.

5.5 Page 5: Event Detail Page (/event/:id)

Displays full information about a single event.

Content: Banner image, title, sport category, description, venue address and city, date and time, available seats, price, organizer name.

Register Button: If the user is not yet registered for this event and seats are available, show a prominent **Register & Pay** button. Clicking it triggers the Razorpay checkout flow. If already registered, show a **You are Registered** badge.

Similar Events Section: Below the main content, display 3–5 event cards of similar events (same sport category and city) as a horizontal row (Feature 5).

5.6 Page 6: Organizer Dashboard (/organizer)

Accessible only to authenticated users with role **organizer**.

Tab 1 – My Events: A table listing all events created by this organizer. Columns: Event Name, Date, Registrations, Seats Remaining, Revenue (INR), Fill Rate (%), Performance Label (LOW / MEDIUM / HIGH shown as a colour badge). There is an action column with Edit and Delete buttons for each event.

Tab 2 – Create Event: A form to create a new event with all Event model fields (title, sport category, description, venue city, venue address, event date, capacity, price, tags). On submission, the event is saved and synced to Algolia.

Tab 3 – Analytics:

- **Registration Trend Graph (Feature 10):** A line chart showing daily registration counts for a selected event. The organizer selects an event from a dropdown to view its trend.
- **Ticket Sales Summary (Feature 11):** For each event, display tickets sold, total capacity, percentage filled, and total revenue as stat cards.
- **Popular Category Insight (Feature 12):** A horizontal bar chart showing which sport categories attract the most registrations across all of the organizer's events.

5.7 Page 7: Admin Dashboard (/admin)

Accessible only to authenticated users with role `admin`.

Tab 1 – Platform Overview (Feature 13): Four large stat cards at the top showing: Total Users, Total Events, Total Registrations, and Total Revenue (INR) across the platform.

Tab 2 – Sport Analytics (Feature 14): A bar chart and a table showing each sport category ranked by total registrations across the platform.

Tab 3 – City Distribution (Feature 15): A pie chart and bar chart showing how many registered users exist in each city.

Tab 4 – Event Fill Rates (Feature 16): A searchable and sortable table of all events on the platform with columns: Event Name, Organizer, Capacity, Seats Filled, Seats Remaining, Fill Rate (%), Status Badge.

Tab 5 – Monthly Trend (Feature 17): A line chart showing the total number of confirmed registrations per calendar month over a rolling 12-month window. Each data point is annotated with month-on-month growth delta.

Tab 6 – Organizer Performance (Feature 18): A ranked table comparing all organizers by total events created, total registrations, and total revenue generated. Top performers are visually highlighted.

Tab 7 – Escalation Tickets: A table of unresolved chatbot escalation tickets. Each row shows the user's original query, timestamp, and a **Resolve** button to mark it as handled.

5.8 Component: AI Chatbot Widget

A floating chat bubble visible on the bottom-right corner of every page for authenticated users (all roles). Clicking the bubble opens a chat modal. The user types a question, the frontend sends it to the chatbot API, and the response appears in the modal. If the chatbot cannot answer, it displays a message saying the query has been escalated to a human agent.

6. API Endpoint Specification

Agent Rule #6: Implement every API endpoint listed in this chapter. All endpoints are prefixed with `/api`. All request and response bodies use JSON. Protected endpoints must verify a valid JWT in the Authorization header as `Bearer <token>`. Role enforcement is as specified in the Auth column.

6.1 Authentication Endpoints

Method	Endpoint	Description	Auth
POST	<code>/api/register</code>	Accept name, email, password, role, city, budget_preference, preferred_sports. Hash the password. Save user. Return JWT and user_id with HTTP 201.	None (Public)
POST	<code>/api/login</code>	Accept email and password. Verify credentials. Return JWT with role claim and HTTP 200. Return 401 if invalid.	None (Public)
GET	<code>/api/me</code>	Return the authenticated user's profile: name, email, role, city, budget_preference, preferred_sports.	Any role
PUT	<code>/api/me</code>	Update the authenticated user's profile fields (city, budget, sports prefs).	Any role

6.2 Event Endpoints

Method	Endpoint	Description	Auth
GET	/api/events	Return a list of all active events. If user is authenticated, apply recommendation scoring (Features 1–4): preferred sports score +4, city match +3, budget match +2, within 7 days +1. Sort descending by total score. For anonymous users, sort by total registration count descending.	Optional JWT
GET	/api/events/:id	Return full details of a single event by ID including organizer name, seats_remaining, fill_rate, and performance_label.	None (Public)
GET	/api/events/:id/similar	Return up to 5 events with the same sport_category and venue_city as the specified event, sorted by registration count descending, excluding the event itself (Feature 5).	None (Public)
POST	/api/events	Create a new event. Accept all Event model fields. Set organizer_id to the JWT identity. Auto-compute price_tier. Sync the new event to Algolia on success. Return the created event with HTTP 201.	Organizer only
PUT	/api/events/:id	Update an existing event. Only the event's owner (or-	Organizer only

6.3 Registration Endpoints

Method	Endpoint	Description	Auth
POST	/api/registrations	Create a new registration for the authenticated user for the given event_id. Check that seats are available. Check the user has not already registered. Set status to pending . Return the registration ID for the payment flow to use.	user only
GET	/api/registrations/my	Return all registrations belonging to the authenticated user, including event details and payment status.	user only
PUT	/api/registrations/:id/cancel	Cancel a registration. Set status to cancelled . Only the owning user may cancel.	user only

6.4 Payment Endpoints

Method	Endpoint	Description	Auth
POST	/api/payments/create-order	Accept event_id. Calculate the amount in paise ($\text{price} \times 100$). Call Razorpay API to create an order. Create a Registration record with status pending and a Payment record with status created . Calculate platform_fee as 15–20% and organizer_payout as remainder. Return order_id, amount, currency, and Razorpay key_id to the frontend.	user only
POST	/api/payments/verify	Accept razor-pay_order_id, razor-pay_payment_id, razor-pay_signature. Verify the HMAC-SHA256 signature using the Razorpay key secret. If valid, update Payment status to paid , update Registration status to confirmed . Return HTTP 200. If signature is invalid, return HTTP 400.	user only

6.5 Chatbot Endpoint

Method	Endpoint	Description	Auth
POST	/api/chatbot	Accept a JSON body with field message (the user's question). Pass the query through the LangChain RAG pipeline. Return the LLM's response as response field in JSON. If the LLM cannot find a good answer (low confidence or fallback triggered), also save an EscalationTicket record and include escalated: true in the response.	None (Public)

6.6 Organizer Dashboard Endpoints

Method	Endpoint	Description	Auth
GET	/api/organizer/dashboard	Return all events owned by the organizer (identified by JWT identity). For each event return: event_id, title, event_date, venue_city, registrations (confirmed count), seats_remaining, revenue (confirmed $\times$ price), fill_rate, performance_label. Feature 8 and Feature 9.	Organizer only
GET	/api/organizer/trend/:event_id	Return daily registration counts for the specified event as an array of objects with day (date string) and count (integer). Only the owning organizer may access this. Feature 10.	Organizer only
GET	/api/organizer/ticket-summary	Return an array of all events owned by the organizer with: title, capacity, seats_sold, percentage_filled, revenue. Feature 11.	Organizer only
GET	/api/organizer/category-insight	Return a grouped array of the organizer's events by sport_category with total	Organizer only

6.7 Admin Analytics Endpoints

Method	Endpoint	Description	Auth
GET	/api/admin/analytics	Return total_users (all users), total_events (all events), total_registrations (confirmed), total_revenue (sum of all payment amounts with status paid). Feature 13.	Admin only
GET	/api/admin/popular-sport	Return all sport categories ranked by number of confirmed registrations. Each item: sport_category, registration_count. Feature 14.	Admin only
GET	/api/admin/city-distribution	Return all cities with their user counts. Each item: city, user_count. Feature 15.	Admin only
GET	/api/admin/fill-rate	Return all events platform-wide with: event_id, title, organizer_name, capacity, seats_filled, seats_remaining, fill_rate. Feature 16.	Admin only
GET	/api/admin/monthly-trend	Return confirmed regis-	Admin only

7. Intelligence Features Specification

Agent Rule #7: All 18 features listed below must be implemented. Features 1–5 are recommendation logic built inside the `GET /api/events` endpoint and the similar events endpoint. Features 8–12 are organizer analytics endpoints with corresponding Vue chart components. Features 13–18 are admin analytics endpoints with corresponding Vue chart and table components.

7.1 User Features (1–7)

7.1.1 Feature 1 – Event Recommendation by Preferred Sports

When the authenticated user has `preferred_sports` set, all events whose `sport_category` is in that list receive a score bonus of 4. Events are then sorted by total score descending. This ensures sport-matched events always appear at the top of the homepage. New users with no preferences see the most-registered events first.

7.1.2 Feature 2 – Nearby Events Recommendation

Events whose `venue_city` matches the authenticated user's `city` field receive a score bonus of 3. This surfaces locally reachable events higher in the list.

7.1.3 Feature 3 – Upcoming Events Recommendation

Events whose `event_date` falls within the next 7 days from the current date receive a score bonus of 1. A dedicated "This Week" section on the homepage filters and displays only these events.

7.1.4 Feature 4 – Budget-Based Recommendation

Events whose `price_tier` matches the user's `budget_preference` receive a score bonus of 2. This ensures users primarily see events within their stated budget range.

7.1.5 Feature 5 – Similar Event Recommendation

On the Event Detail page, after the main event content, query all other active events with the same `sport_category` and `venue_city`. Sort by confirmed registration count descending. Display the top 3–5 as a horizontal scrollable row of event cards.

7.1.6 Feature 6 – Razorpay Payment Gateway

When a user clicks Register & Pay on an event detail page, the frontend calls `POST /api/payments/create-order`. The backend creates a Razorpay order and returns the order details. The frontend opens the Razorpay checkout modal with these details. After the user completes payment, Razorpay returns three values: `order_id`, `payment_id`, and signature. The frontend immediately calls `POST /api/payments/verify` with these three values. The backend verifies the HMAC signature, then marks the registration as confirmed and the payment as paid. The platform retains 15–20% as `platform_fee` and the remainder is logged as `organizer_payout`.

7.1.7 Feature 7 – AI Chatbot

A floating chat widget is available on all pages. The chatbot uses LangChain with a FAISS vector store of FAQ documents. For each user message, the backend retrieves the top 3 matching FAQ chunks and injects them as context into the LLM prompt. The LLM (OpenAI GPT-4o-mini as primary) generates an answer. If the answer confidence is low or no relevant FAQ is found, the chatbot replies with a fallback message and saves an EscalationTicket to the database for admin review.

7.2 Organizer Features (8–12)

7.2.1 Feature 8 – Organizer Event Dashboard

The organizer’s main dashboard tab shows a table of all their events. Each row displays: event title, date, confirmed registration count, seats remaining, revenue generated, and fill rate percentage. This gives organizers an at-a-glance health check of their entire event portfolio.

7.2.2 Feature 9 – Event Performance Label

Alongside every event in the organizer dashboard, display a coloured badge indicating performance level. A fill rate below 30% shows a red **LOW** badge. Between 30% and 70% shows an amber **MEDIUM** badge. Above 70% shows a green **HIGH** badge.

7.2.3 Feature 10 – Registration Trend Graph

In the organizer analytics tab, a line chart visualises how registrations have accumulated for a specific event over time. The organizer selects an event from a dropdown, and the chart updates to show daily registration counts. This helps identify marketing spikes and slow periods.

7.2.4 Feature 11 – Ticket Sales Summary

A set of stat cards for each organizer event showing: total tickets sold, total event capacity, percentage filled (computed as $\text{sold} \div \text{capacity} \times 100$), and total revenue collected.

7.2.5 Feature 12 – Popular Category Insight

A horizontal bar chart showing which sport categories attract the most registrations across all events created by this organizer. The X-axis shows registration counts and the Y-axis lists sport categories.

7.3 Admin Features (13–18)

7.3.1 Feature 13 – Platform Analytics Dashboard

Four large stat cards at the top of the admin dashboard showing: total number of registered users, total number of events, total confirmed registrations across the platform, and total revenue collected.

7.3.2 Feature 14 – Most Popular Sport Category

A bar chart and a ranked list showing every sport category on the platform ordered by total confirmed registrations. The top sport category is highlighted.

7.3.3 Feature 15 – City-Wise User Distribution

A pie chart and a table showing how users are distributed across cities. Each entry shows city name and the number of registered users from that city.

7.3.4 Feature 16 – Event Fill-Rate Dashboard

A table listing every active event on the platform, sortable by fill rate. Admins can identify events that are nearly sold out (and may need capacity expansion) or events that are underperforming (and may need promotion).

7.3.5 Feature 17 – Monthly Registration Trend

A line chart covering the past 12 calendar months. Each point on the chart represents the total number of confirmed registrations in that month. A delta annotation below each point shows the month-on-month percentage change.

7.3.6 Feature 18 – Organizer Performance Comparison

A ranked table comparing all organizers on the platform by three metrics: total events created, total confirmed registrations, and total revenue generated. Top-performing organizers are visually highlighted to assist with partnership decisions.

8. Algolia Search Specification

Agent Rule #8: Every time an event is created or updated, sync the event record to Algolia. Every time an event is deleted (soft-deleted), remove it from the Algolia index. The Vue frontend must use the Algolia JavaScript client directly for search — do not route search queries through the Flask API.

8.1 Algolia Index Schema

The Algolia index named `events` must store the following fields for each event object:

Algolia Field	Source
objectID	Event.id (as string)
title	Event.title
sport_category	Event.sport_category
venue_city	Event.venue_city
event_date	Event.event_date (ISO string)
price	Event.price
price_tier	Event.price_tier
tags	Event.tags

8.2 Search Configuration

Configure Algolia searchable attributes as: title, sport_category, venue_city, tags. Configure custom ranking to sort by event_date ascending as a secondary rank after relevance.

8.3 Frontend Search Behaviour

The Algolia search bar on the User Home page must be a live search component. As the user types, the Algolia client performs an instant search and renders matching event cards in real time. The search supports typo tolerance, meaning small spelling errors still return relevant results.

9. Payment Flow Specification

Agent Rule #9: Implement the Razorpay payment flow exactly as described in the five steps below. Do not allow a user to gain a confirmed registration without a verified payment signature.

9.1 End-to-End Payment Flow

The complete flow from user intent to confirmed registration is:

1. The user views an Event Detail page and clicks **Register & Pay**.
2. The Vue frontend sends a POST request to `/api/payments/create-order` with the `event_id`. The backend creates a Razorpay order via the Razorpay Python SDK, creates a Registration record (status: pending), and a Payment record (status: created). The backend returns `order_id`, `amount`, `currency`, and `key_id` to the frontend.
3. The frontend opens the Razorpay checkout modal using the returned values. The user completes payment via their preferred method (card, UPI, wallet, or net banking).
4. Razorpay calls the frontend handler with three values: `razorpay_order_id`, `razorpay_payment_id`, and `razorpay_signature`.
5. The frontend immediately sends a POST request to `/api/payments/verify` with the three Razorpay values. The backend recomputes the HMAC-SHA256 signature using the secret key and verifies it matches. On success, the Payment status is updated to **paid** and the Registration status is updated to **confirmed**. The `organizer_payout` is stored as the amount minus the platform fee (15–20%). The user sees a success message.

9.2 Platform Fee Calculation

The platform fee percentage is stored in the application config as `PLATFORM_FEE_PERCENT` (default 0.15, representing 15%). The `organizer_payout` stored in the Payment record equals:

$$\text{organizer_payout} = \text{amount} \times (1 - \text{PLATFORM_FEE_PERCENT})$$

10. AI Chatbot Specification

Agent Rule #10: Build the chatbot as a LangChain Retrieval-Augmented Generation (RAG) pipeline. The FAISS vector store must be initialised at application startup with the FAQ documents listed below. Do not regenerate embeddings on every request — cache the vector store in memory.

10.1 RAG Pipeline Architecture

When a user sends a message to the chatbot endpoint, the following steps occur in sequence inside the backend service:

1. The user's query is received as plain text.
2. The LangChain retriever searches the FAISS vector store for the top 3 most semantically similar FAQ chunks.
3. The retrieved chunks are injected as context into a PromptTemplate alongside the user's question.
4. The assembled prompt is sent to the LLM (OpenAI GPT-4o-mini).
5. The LLM generates an answer grounded in the FAQ context.
6. The answer is returned to the user via the API response.
7. If no relevant FAQ context is found (retrieval score is below threshold), the chatbot responds with a standard escalation message and saves an EscalationTicket record to the database.

10.2 Initial FAQ Knowledge Base

The vector store must be seeded with at least the following FAQ entries at application startup. More can be added over time via a database-backed FAQ management system:

- How do I register for an event? Navigate to the event detail page and click Register & Pay. Complete the Razorpay checkout to confirm your spot.
- What payment methods are accepted? You can pay via credit card, debit card, UPI, mobile wallets, and net banking through Razorpay.
- How do I cancel my registration? Go to My Registrations and click Cancel on the event. Refund policies depend on the organizer.

- How do I get a refund? Submit a refund request within 48 hours of cancellation. Refunds are processed within 5–7 business days.
- Where can I see my registered events? After logging in, go to My Registrations from your profile menu.
- How do I contact support? Use this chatbot for instant answers. For unresolved issues, our team will follow up within 24 hours.
- Can I register multiple people for one event? Currently each account registers one participant. Create separate accounts for multiple participants.
- What happens if an event is cancelled? You will receive a full refund and an email notification.
- How do organizers receive payouts? Razorpay processes organizer payouts automatically after the platform fee is deducted.
- How do I become an organizer? Select the Organizer role during registration.

11. Folder Structure & Project Setup

11.1 Backend Directory Layout

The backend project must follow this structure:

```
backend/  
app/ — Main application package  
__init__.py — Application factory function (create_app)  
config.py — DevelopmentConfig and ProductionConfig classes  
extensions.py — Singleton instances of db, jwt, cors, migrate  
models/ — One file per model: user.py, event.py, registration.py, payment.py, esca-  
lation.py  
api/ — One Blueprint file per group: auth.py, events.py, registrations.py, pay-  
ments.py, chatbot.py, organizer.py, admin.py  
services/ — Business logic layer: recommendation.py, algolia_sync.py, razorpay_service.py,  
chatbot_service.py  
utils/ — Shared helpers: decorators.py (role_required), helpers.py  
migrations/ — Flask-Migrate auto-generated files  
tests/ — Unit and integration tests  
requirements.txt — All Python dependencies pinned  
.env — Environment variables (not committed to git)  
run.py — Entry point that calls create_app()
```

11.2 Frontend Directory Layout

The frontend project must follow this structure:

```
frontend/  
src/  
main.js — App entry, mounts Vue, registers router and Pinia  
App.vue — Root component with router-view and chatbot widget  
router/index.js — All route definitions with role guards  
stores/ — auth.js (token, role, userId), events.js, chatbot.js  
views/ — LandingPage.vue, LoginPage.vue, RegisterPage.vue, HomePage.vue, Event-  
Detail.vue, OrganizerDashboard.vue, AdminDashboard.vue  
components/ — EventCard.vue, RecommendationRow.vue, ChatbotWidget.vue  
components/charts/ — RegistrationTrend.vue, CategoryBarChart.vue, FillRateTable.vue,
```

MonthlyTrendChart.vue, CityDistributionChart.vue
components/payment/ — RazorpayCheckout.vue
services/api.js — Axios instance with base URL and auth header interceptor
services/algolia.js — Algolia search client and searchEvents function
.env — VITE_ prefixed environment variables
package.json — All npm dependencies
vite.config.js — Vite configuration

11.3 Environment Variables

The following environment variables must be defined in the **.env** file at the backend root. They must never be hardcoded in any source file:

Variable	Purpose
SECRET_KEY	Flask secret key for session signing
DATABASE_URL	PostgreSQL connection string (production)
RAZORPAY_KEY_ID	Razorpay public key ID
RAZORPAY_KEY_SECRET	Razorpay secret key for HMAC verification
PLATFORM_FEE	Decimal fee fraction, e.g. 0.15
ALGOLIA_APP_ID	Algolia application ID
ALGOLIA_API_KEY	Algolia admin API key (backend)
ALGOLIA_INDEX	Algolia index name (e.g. events)
OPENAI_API_KEY	OpenAI API key for chatbot LLM
GEMINI_API_KEY	Google Gemini API key (optional fallback)
HUGGINGFACE_TOKEN	HuggingFace inference API token (optional fallback)

The following VITE_ prefixed variables must be defined in the frontend **.env** file:

Variable	Purpose
VITE_API_BASE_URL	Backend API base URL, e.g. http://localhost:5000
VITE_ALGOLIA_APP_ID	Algolia application ID
VITE_ALGOLIA_SEARCH_KEY	Algolia search-only API key (public safe)
VITE_ALGOLIA_INDEX	Algolia index name
VITE_RAZORPAY_KEY_ID	Razorpay public key ID (for frontend modal)

12. Implementation Sprint Plan

12.1 Sprint Overview

All 18 features are estimated at approximately 20 developer-days total, to be completed within a 4-week sprint:

Week	Deliverables
Week 1	Backend: Models, migrations, auth API (register/login/me). Frontend: Landing Page, Register Page, Login Page, Vue Router setup, Pinia auth store, role-based redirect after login. Backend: Events CRUD API, Recommendation service (Features 1–4), Similar events endpoint (Feature 5). Frontend: Home page with recommendation sections, Event Detail page with similar events row.
Week 2	Backend: Razorpay payment endpoints (create-order, verify) (Feature 6). Frontend: Razorpay checkout component, registration flow on Event Detail page. Backend: Organizer dashboard endpoints (Features 8–12). Frontend: Organizer Dashboard page with all tabs, Chart.js components (line chart, bar chart, stat cards).
Week 3	Backend: Admin analytics endpoints (Features 13–18) and escalation ticket endpoints. Frontend: Admin Dashboard page with all tabs, city distribution chart, monthly trend chart, organizer performance table, escalation ticket table. Algolia integration: Backend sync service, frontend Algolia search bar on Home page.
Week 4	Backend: LangChain chatbot service with FAISS vector store and RAG pipeline (Feature 7). Frontend: Floating chatbot widget component. QA: End-to-end testing of all flows (auth, payment, recommendations, dashboards, chatbot). Deployment: Backend to hosting platform (Render / EC2), frontend to Vercel / Netlify, PostgreSQL database migration, Algolia index seeding.

12.2 Feature Effort Summary

#	Feature Name	Role	Complexity	Days
1	Event Recommendation (Sports)	User	Low	1.0
2	Nearby Events (City)	User	Low	1.0
3	Upcoming Events (7-day)	User	Low	0.5
4	Budget-Based Filter	User	Low	1.0

5	Similar Event Recommendation	User	Low	1.0
6	Razorpay Payment Gateway	User	Medium	3.0
7	AI Chatbot (LangChain + RAG)	All Users	Medium	3.0
8	Organizer Event Dashboard	Organizer	Low	1.0
9	Event Performance Label	Organizer	Low	1.0
10	Registration Trend Graph	Organizer	Low	1.0
11	Ticket Sales Summary	Organizer	Low	1.0
12	Popular Category Insight	Organizer	Low	1.0
13	Platform Analytics Dashboard	Admin	Low	1.0
14	Most Popular Sport Category	Admin	Low	0.5
15	City-Wise User Distribution	Admin	Low	1.0
16	Event Fill-Rate Dashboard	Admin	Low	1.0
17	Monthly Registration Trend	Admin	Low	1.0
18	Organizer Performance Compare	Admin	Low	1.0
Total Estimated Effort				≈ 20 days

13. Hosting & Deployment

13.1 Hosting Options

Component	Option A – Free / Low Cost	Option B – Production Grade
Backend API	Render.com (free tier, sleeps after inactivity)	AWS EC2 or GCP Cloud Run (always on)
Database	Render PostgreSQL (free tier, 1 GB limit)	AWS RDS or Supabase (managed, backups)
Frontend	Vercel or Netlify (free tier, auto-deploy from GitHub)	Vercel Pro or AWS S3 + CloudFront (CDN)
Search	Algolia Free (10k records, 10k searches/month)	Algolia Grow (higher limits)
Payments	Razorpay Test Mode (no real money)	Razorpay Live Mode (real transactions)

13.2 Deployment Checklist

Before going live, complete the following steps in order:

1. Set all environment variables in the hosting platform's secret manager.
2. Run `flask db upgrade` against the production PostgreSQL database to apply all migrations.
3. Seed the Algolia index by triggering the sync service for all existing events.
4. Set the Razorpay account to Live Mode and update the key variables.
5. Deploy the Vue frontend with the production API base URL configured.
6. Run a smoke test on all critical flows: register, login, browse events, register for event, pay, view organizer dashboard, view admin dashboard, use chatbot.

Conclusion & Agent Summary

This document is the complete specification for the Sports Event Management MVP. It defines every model, relationship, API endpoint, UI page, intelligence feature, payment flow, search integration, and AI chatbot behaviour required to build the full application.

Coding Agent Final Checklist:

- Landing page is the root route / and is fully public.
- Login and Register buttons are on the landing page navbar.
- All three roles (user, organizer, admin) log in through the same login page.
- Post-login redirect is role-based using the JWT role claim.
- All 5 database models are created with correct fields and foreign keys.
- All 26 API endpoints are implemented with correct HTTP methods and auth.
- Role-based access control is enforced on every protected endpoint.
- All 18 intelligence features are implemented and functional.
- Razorpay payment flow is complete: order creation → checkout → verification.
- Algolia is synced on event create/update/delete and searched on the frontend.
- LangChain RAG chatbot is operational with FAISS vector store.
- All environment variables are loaded from .env and never hardcoded.

Prepared by: SE Team 026 – Syntax Error / February 17, 2026 / Version 1.0